Sketch-Based 3D Shape Generation: Project of Computer Vision and Computer Graphics Integration

Abstract

This project demonstrates a real-time system that transforms the user's 2D hand-drawn sketch into a 3D model using shape-detection, extrusion and surface-of-revolution techniques. The system uses OpenCV contour extraction and OpenGL rendering to provide an interactive modelling pipeline. The full implementation is based on C++, and with the help of the earlier mentioned libraries which include real-time drawing capture, shape recognition, mesh generation, and GPU visualization of the extracted and transformed sketches. The goal is to demonstrate how computer vision and computer graphics can be integrated into one system to build intuitive shape-generation tools.

Problem statement and motivation

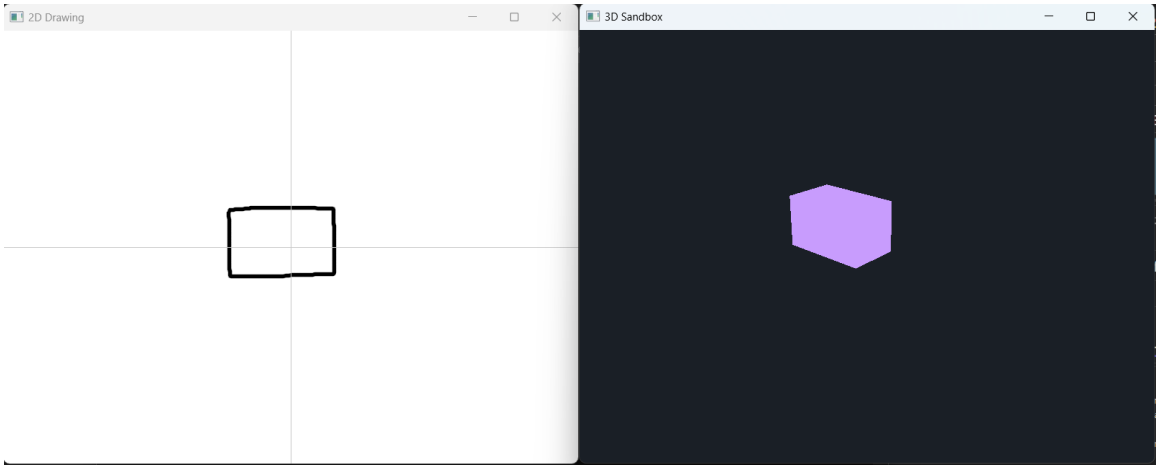
Sketch-based modelling can be viewed as an intuitive connection between the human creativity and the phenomenon of computational geometry. The challenging part is to reliably transform 2D sketches (input made by user) into any valid 3D geometry while maintaining the real-time performance and the ease of interaction with the created objects and UI windows.
In the field of Visual Computing, this project connects two major subfields:

• Computer Vision — used in order to interpret and analyse 2D sketch by the user, extract contours, detect polygons, and classify basic shapes.
• Computer Graphics — used in order to construct, manipulate, and render 3D geometry created the from sketches.

The objective is to design and implement a system where users can draw on a 2D canvas (one window), and the software automatically converts these drawings into meaningful 3D objects and set them in a 3D space (other window) using multiple methods for the construction. This can provide facility for rapid idea creation and for example early-stage modelling of any product.

System overview

This project implements a dual-window application containing a sketch interface (one window, white background) and a 3D viewer and manipulator window (other window, black background). When the sketch window is in the focus the users can draw strokes, trigger shape recognition by pressing specified keys, and generate meshes using extrusion, revolution or shape detecting (contour extraction). The system combines OpenCV for image analysis and OpenGL for 3D rendering.



1. figure: 2 windows, one for sketching, one for 3d space

Implementation

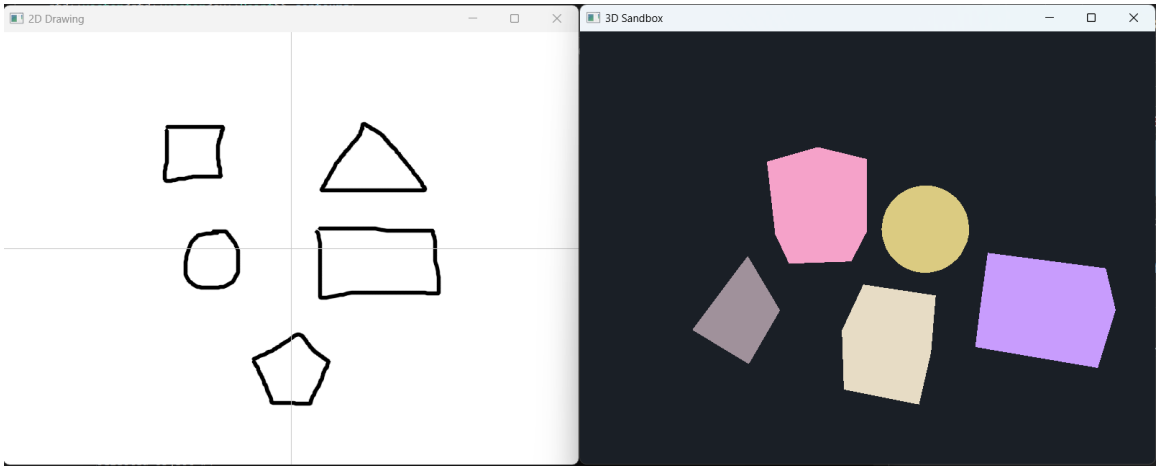
Capturing input and sketch processing

The system captures user strokes in a GLFW-created window. The mouse callbacks create strokes on a white OpenCV canvas, which fills the window to give a reasonable amount of drawing space, where each stroke is drawn as a polyline onto the canvas. The image is then processed using thresholding, contour detection, and polygon approximation:

- OpenCV functions are used to extract contours, such as:
    - `cv::threshold`
    - `cv::findContours`
    - `cv::approxPolyDP`
- The largest detected contour is interpreted as the user's intended shape.
- The canvas can be cleared to start over the drawing and to recognize new shape without running out of room to draw
- Two types of representation are extracted:
    - An information about polygon for shape recognition (string representation of recognized shape and points of shape).
    - A raw stroke polyline for extrusion and revolution (points of shape).

Shape Recognition

By supporting predefined shapes (cube, cuboid, sphere, pyramid, pentagonal prism), the system classifies contours based on the drawn polygon's vertex count, aspect ratios, and circularity measure. Based on the classification (can classify: square, rectangle, circle, triangle, pentagon), a corresponding pre-generated mesh is inserted into the 3D scene. Shape detection is triggered using the "R" key (2D window is in focus).



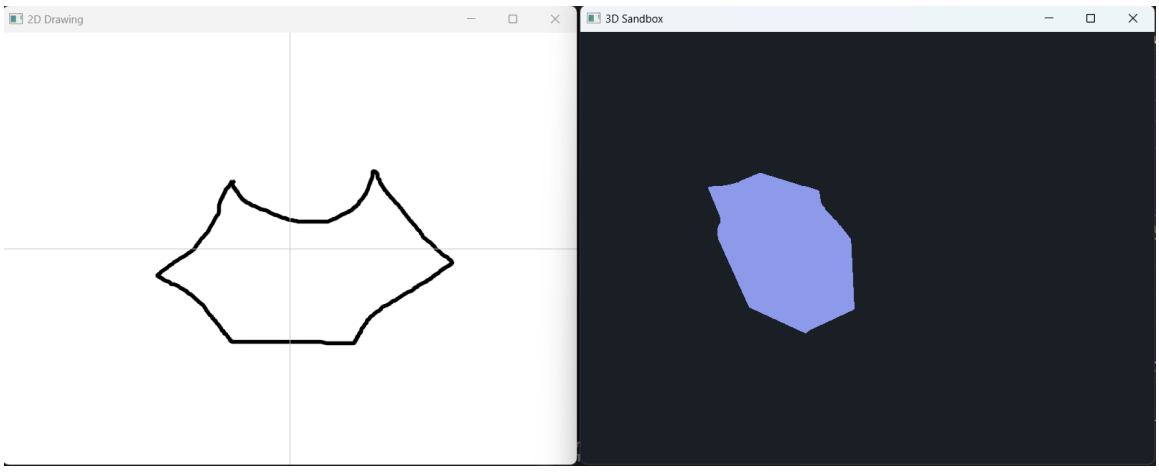
2. figure: 2D shapes and their corresponding 3D objects

Extrusion algorithm

The extrusion algorithm transforms a 2D polygon into a 3D prism by creating another parallel copy of the polygon from the original polygon in a predefined distance using the y axis and connects them with faces. The implementation performs the following steps:

1. Normalize and center polygon coordinates.
2. Generate top and bottom vertex rings.
3. Create indices for the polygon caps.
4. Construct wall faces between corresponding edges.

Extrusion is triggered using the “X” key in the application (2D window is in focus).



3. figure: extrusion algorithm result with input

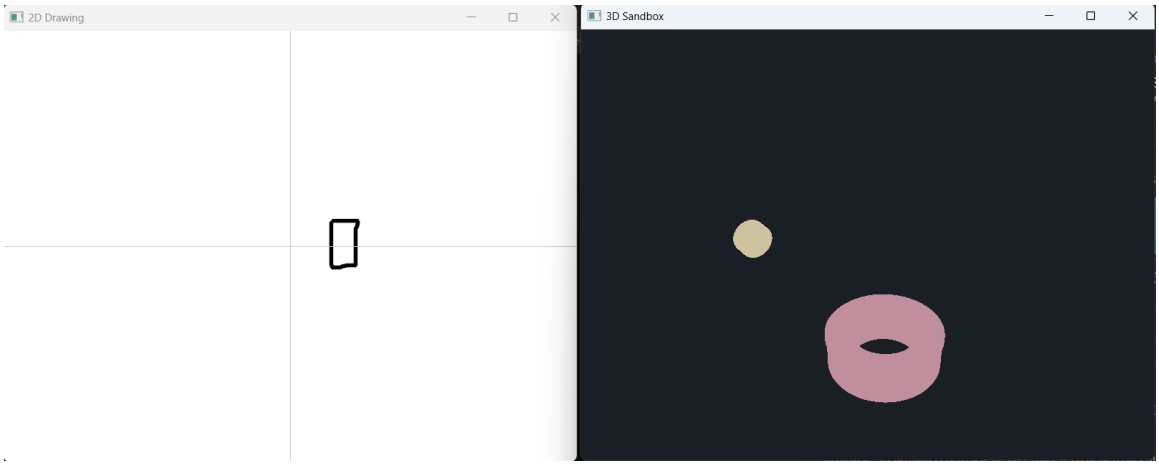
Revolution algorithm

The system also enables another type of 3D construction, where a raw 2D stroke is rotated around the X- or Y-axis to generate a 3D surface. This is essential for creating objects like vases, bowls, handles. wheel-like objects, etc.

Steps:

1. Convert stroke into centered coordinates.
2. Sample the stroke along the axis.
3. For each angular segment, apply rotation using trigonometric transformations.
4. Connect segments to form a closed mesh.

These operations are executed when users press the “V” key (X-axis revolution) or “H” key (Y-axis revolution) (2D window is in focus).



4. figure: revolution of the same sketch with both axes

GPU Mesh Upload and Rendering

Once a mesh is generated, each of them is sent to the GPU using OpenGL VBOs and EBOs. The system also computes all of the mesh’s bounding boxes for interaction.
The rendering uses a simple shader pipeline which supports unique colors for objects to distinguish and standard transformations. Objects can be rotated, translated, or scaled via mouse ray - object interaction enabled by the 3D mouse control.

Interaction and Object Manipulation

When the 3D window has the focus, users can alternate between camera navigation mode and object manipulation mode (using the “T” key). When manipulating objects, ray casting is used to select meshes, and dragging operations modify the position (left click hold during object manipulation mode), the angle (right click hold during object manipulation mode), and the scale

(scroll during object manipulation mode) factor of the selected object. This interactive workflow turns the system into a lightweight modelling tool. Moving in the 3D space is possible by holding the right mouse button and using the “ASDW” for moving in the classical way. By pressing the “E” and “Q” keys while holding the right mouse button the use can move up and down straight. When moving forward and backward the mouse placed cursor alters the movement’s direction into the direction of the ray (where the user faces in the 3D space).

Experimental Evaluation and Analysis

Qualitative evaluation

The quantitative evaluation was in the form of a short survey where the users had to create the following object (a tractor):

The rules were:

- A brick-like body
- 4 wheels, where the wheels’ middle is not filled
- An object on top of body to imitate driver’s cabin
- Object placement has to be realistic

5. figure: the scene that the survey takers had to recreate

The survey takers were separated into 2 groups: (3-3 members)

- Group 1: has previous knowledge from IT or 3D rendering / design
- Group 2: has very limited on no prior knowledge of IT or 3D rendering / design

The survey takers had to express their opinion about the system’s intuitiveness, visual result’s quality, recognition’s precision. The following results emerged:

- Group 1: Overall, the system was easy to use, they get the hang of it in a short time, the visual results were pleasurable and close to the original scene. At first because of lousy sketches, the recognition was not easy to use, then the users tried to draw regular polygons to improve the recognition. They had an average time of 1 minute and 15 seconds to complete.

- Group 2: Overall, the system was a little harder to interpret because they lack experience with 3D systems. However, the results were similarly good compared to Group 1. Recognition had no issues as they tried to formulate close to regular polygon shapes. They had an average time of 2 minutes and 11 seconds.

Conclusion

The system was easy-to-learn and easy-to-use as well. For those who had worked with 3D system before the controls made sense as they were modelled from existing solutions (Unity). For those who did not work with 3D prior, showed a steep learning curve and quality visual results. That's why the system is usable for both kind of people (IT / non-IT).

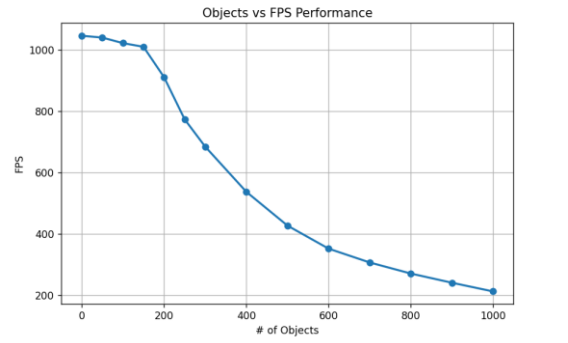
Quantitative evaluation

System used for testing

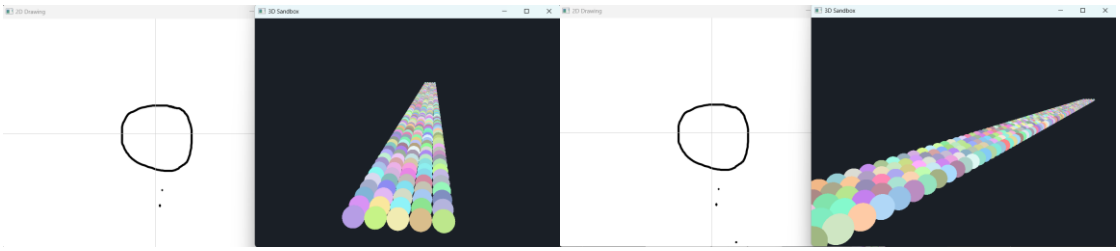
CPU	*AMD Ryzen 7 7735HS*
GPU	*NVIDIA GeForce RTX 4050 Laptop GPU*
RAM	*16 GB RAM*
Development environment	*2022 Visual Studio IDE, Windows SDK – Windows 10.0.26100.1*

of objects vs. FPS

This analysis measures how the FPS is affected by adding more and more objects to the scene. For the analysis the same sphere has been inserted into the scene and measured and average FPS (under the time span of 20 seconds) when the number of objects reached the measure point.



6. *figure: graph of #of objects vs. FPS*



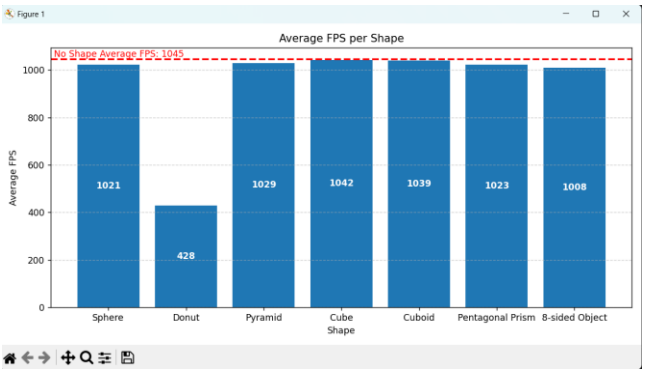
7. figure: Scene during analysis

Observation

The FPS drops gradually as the number of objects increase but not linearly, rather in a polynomial way. The system even when using maximum capacity, still maintained an FPS rate of 200+ which resulted in still smooth interaction in the 3D space.

Object complexity vs. FPS

In this analysis, 7 types of objects were inserted into the scene with 100 instances. It measures how the FPS drops due to the different complexity of the objects to render. Once the 100 instances were inserted, the average FPS was measured under a 10 second timespan. The Donut was created by revolution, the 8-sided object with extrusion, every other was made by shape recognition.



8. figure: bar chart of average FPS per shape

Observation

The most difficult shape to the pipeline was the donut as there the FPS drastically dropped. The other shapes had similar results. with 1000+ FPS. Another observation is that when a similar object is given but the more side/face the pipeline has to render, the less FPS will it produce.

Discussion

The implemented system demonstrates a functioning and effective pipeline that incorporates computer vision input analysis with 3D graphics. Its features resemble commercial sketch-based

modelling tools. Several strengths are notable:

- Real-time interaction
- Shape recognition
- Two 3D generation techniques over shape detection (extrusion + revolution)
- Direct manipulation of objects with easy-to-learn interactions

The system's limitations are:

- Sensitivity to noisy sketches
- Lack of smoothing for raw strokes
- Absence of mesh optimization
- Maximum number of mashes specified

These limitations point to natural extensions for future work.

Conclusion

This project shows the cooperation between computer vision and computer graphics by creating intuitive 3D modelling interfaces. By combining the OpenCV-based contour processing with the OpenGL rendering, the system allows the users to create simple sketches and then manipulate the 3D objects derived from the sketch. This approach provides a foundation for more sophisticated modelling tools, such as neural sketch-based modelling (instead of contour extraction), or geometry processing with physics awareness.

Task-per-Member

Emmanouil Platakis: 3D window and interactions (moving of camera+ object manipulation, connection of 2D with 3D window, report writing, survey conduction (Group 2), qualitative analysis – complexity of objects vs FPS

Kornidesz Máté: 2D window and 3 types of object creation, connection of 2D window with 3D window, report writing, survey conduction (Group 1), qualitative analysis - # of objects vs FPS